

Punto 4.2 - Tracking y No Tracking

Práctica

Ejercicio: Añadir métodos al repositorio del proyecto AceriaData que demuestren el comportamiento del tracking y del no tracking. Comparar el número de entidades rastreadas, el tiempo de ejecución y el SQL generado. Aplicar AsNoTracking en consultas de solo lectura y Tracking en consultas de escritura.

Contexto del proyecto: En el punto 4.1 se introdujo el análisis del SQL generado con ToQueryString y logging. En este punto se profundiza en el control del tracking, que es una de las técnicas de optimización más importantes en EF Core. Esta técnica se usará en el punto 4.3 para las variantes de No Tracking.

Paso 1: Abrir el proyecto

```
cd AceriaData
cd src/AceriaData.Console
```

cd AceriaData → entra en la carpeta raíz del proyecto. cd src/AceriaData.Console → entra en la carpeta del proyecto de consola.

Paso 2: Añadir los métodos de tracking a la interfaz del repositorio

Modificar src/AceriaData.Application/Interfaces/IOrdenRepositorio.cs:

```
using AceriaData.Application.Dtos;
using AceriaData.Domain.Entities;

namespace AceriaData.Application.Interfaces;

public interface IOrdenRepositorio
{
    // ... métodos existentes ...

    List<OrdenFabricacion> ObtenerConTracking();
    List<OrdenFabricacion> ObtenerSinTracking();
    int ContarEntidadesRastreadas();

    void Agregar(OrdenFabricacion orden);
    void Eliminar(OrdenFabricacion orden);
}
```

Línea 1: using AceriaData.Application.Dtos; → importa los DTOs.

Línea 2: using AceriaData.Domain.Entities; → importa las entidades.

Línea 4: namespace AceriaData.Application.Interfaces; → declara el espacio de nombres.

Línea 6: public interface IOrdenRepositorio → declara la interfaz.

Línea 8: // ... métodos existentes ... → comentario que indica que se mantienen los métodos anteriores.

Línea 10: List ObtenerConTracking(); → método que obtiene las órdenes con Tracking.

Línea 11: List ObtenerSinTracking(); → método que obtiene las órdenes sin Tracking.

Línea 12: int ContarEntidadesRastreadas(); → método que cuenta las entidades rastreadas.

Línea 14: void Agregar(OrdenFabricacion orden); → método que agrega una orden.

Línea 15: void Eliminar(OrdenFabricacion orden); → método que elimina una orden.

Error común: si los métodos devuelven IQueryable, la capa de aplicación puede componer consultas que el repositorio no controla. Se deben devolver listas o valores concretos.

Paso 3: Implementar los métodos de tracking en el repositorio

Modificar src/AceriaData.Infrastructure/Repositories/OrdenRepository.cs:

```
public List<OrdenFabricacion> ObtenerConTracking()
{
    return _context.OrdenesFabricacion
        .OrderBy(o => o.NumeroOrden)
        .ToList();
}

public List<OrdenFabricacion> ObtenerSinTracking()
{
    return _context.OrdenesFabricacion
        .AsNoTracking()
        .OrderBy(o => o.NumeroOrden)
        .ToList();
}

public int ContarEntidadesRastreadas()
{
    return _context.ChangeTracker.Entries().Count();
}
```

Línea 1: public List ObtenerConTracking() → declara el método.

Línea 3: return _context.OrdenesFabricacion → inicia la consulta.

Línea 4: .OrderBy(o => o.NumeroOrden) → ordena por número de orden.

Línea 5: .ToList(); → materializa la consulta con Tracking.

Línea 8: public List ObtenerSinTracking() → declara el método.

Línea 10: return _context.OrdenesFabricacion → inicia la consulta.

Línea 11: .AsNoTracking() → aplica AsNoTracking.

Línea 12: .OrderBy(o => o.NumeroOrden) → ordena por número de orden.

Línea 13: .ToList(); → materializa la consulta sin Tracking.

Línea 16: public int ContarEntidadesRastreadas() → declara el método.

Línea 18: return _context.ChangeTracker.Entries().Count(); → cuenta las entidades rastreadas.

Error común: si se llama a ContarEntidadesRastreadas después de ObtenerSinTracking, el número es cero porque las entidades no se han registrado.

Paso 4: Crear el caso de uso de tracking

Crear el archivo src/AceriaData.Application/UseCases/TrackingUseCase.cs:

```
using System.Diagnostics;
using AceriaData.Application.Interfaces;

namespace AceriaData.Application.UseCases; public class TrackingUseCase
{
    private readonly IUnidadDeTrabajo _unidad;

    public TrackingUseCase(IUnidadDeTrabajo unidad)
    {
        _unidad = unidad;
    }

    public void Ejecutar()
    {
        Console.WriteLine("=== TRACKING Y NO TRACKING ===");

        DemostrarConTracking();
    }
}
```

```

        DemostrarSinTracking();
        CompararRendimiento();
    }

    private void DemostrarConTracking()
    {
        Console.WriteLine("\n--- Con Tracking ---");

        var ordenes = _unidad.Ordenes.ObtenerConTracking();
        var rastreadas = _unidad.Ordenes.ContarEntidadesRastreadas();

        Console.WriteLine($"Órdenes cargadas: {ordenes.Count}");
        Console.WriteLine($"Entidades rastreadas: {rastreadas}");
    }

    private void DemostrarSinTracking()
    {
        Console.WriteLine("\n--- Sin Tracking ---");

        var ordenes = _unidad.Ordenes.ObtenerSinTracking();
        var rastreadas = _unidad.Ordenes.ContarEntidadesRastreadas();

        Console.WriteLine($"Órdenes cargadas: {ordenes.Count}");
        Console.WriteLine($"Entidades rastreadas: {rastreadas}");
    }

    private void CompararRendimiento()
    {
        Console.WriteLine("\n--- Comparación de rendimiento ---");

        var cronometroCon = Stopwatch.StartNew();
        var ordenesCon = _unidad.Ordenes.ObtenerConTracking();
        cronometroCon.Stop();

        var cronometroSin = Stopwatch.StartNew();
        var ordenesSin = _unidad.Ordenes.ObtenerSinTracking();
        cronometroSin.Stop();

        Console.WriteLine($"Con Tracking: {cronometroCon.ElapsedMilliseconds} ms | Órdenes: {ordenesCon.Count}");
        Console.WriteLine($"Sin Tracking: {cronometroSin.ElapsedMilliseconds} ms | Órdenes: {ordenesSin.Count}");
    }
}

```

Línea 1: using System.Diagnostics; → importa Stopwatch.

Línea 2: using AceriaData.Application.Interfaces; → importa las interfaces de la aplicación.

Línea 4: namespace AceriaData.Application.UseCases; → declara el espacio de nombres.

Línea 6: public class TrackingUseCase → declara el caso de uso.

Línea 8: private readonly IUnidadDeTrabajo _unidad; → campo de la unidad de trabajo.

Línea 10: public TrackingUseCase(IUnidadDeTrabajo unidad) → constructor.

Línea 12: _unidad = unidad; → asigna el parámetro al campo.

Línea 15: public void Ejecutar() → declara el método principal.

Línea 17: Console.WriteLine("=== TRACKING Y NO TRACKING ==="); → muestra la cabecera.

Línea 19: DemostrarConTracking(); → llama al método de Tracking.

Línea 20: DemostrarSinTracking(); → llama al método de No Tracking.

Línea 21: CompararRendimiento(); → llama al método de comparación.

Línea 24: private void DemostrarConTracking() → declara el método.

Línea 26: Console.WriteLine("\n--- Con Tracking ---"); → muestra la cabecera.

Línea 28: `var ordenes = _unidad Ordenes.ObtenerConTracking();` → carga las órdenes con Tracking.

Línea 29: `var rastreadas = _unidad Ordenes.ContarEntidadesRastreadas();` → cuenta las entidades rastreadas.

Línea 31: `Console.WriteLine($"Órdenes cargadas: {ordenes.Count}");` → muestra el número de órdenes.

Línea 32: `Console.WriteLine($"Entidades rastreadas: {rastreadas}");` → muestra el número de entidades rastreadas.

Línea 35: `private void DemostrarSinTracking()` → declara el método.

Línea 37: `Console.WriteLine("\n--- Sin Tracking ---");` → muestra la cabecera.

Línea 39: `var ordenes = _unidad Ordenes.ObtenerSinTracking();` → carga las órdenes sin Tracking.

Línea 40: `var rastreadas = _unidad Ordenes.ContarEntidadesRastreadas();` → cuenta las entidades rastreadas.

Línea 42: `Console.WriteLine($"Órdenes cargadas: {ordenes.Count}");` → muestra el número de órdenes.

Línea 43: `Console.WriteLine($"Entidades rastreadas: {rastreadas}");` → muestra el número de entidades rastreadas.

Línea 46: `private void CompararRendimiento()` → declara el método.

Línea 48: `Console.WriteLine("\n--- Comparación de rendimiento ---");` → muestra la cabecera.

Línea 50: `var cronometroCon = Stopwatch.StartNew();` → inicia el cronómetro para la consulta con Tracking.

Línea 51: `var ordenesCon = _unidad Ordenes.ObtenerConTracking();` → carga las órdenes con Tracking.

Línea 52: `cronometroCon.Stop();` → detiene el cronómetro.

Línea 54: `var cronometroSin = Stopwatch.StartNew();` → inicia el cronómetro para la consulta sin Tracking.

Línea 55: `var ordenesSin = _unidad Ordenes.ObtenerSinTracking();` → carga las órdenes sin Tracking.

Línea 56: `cronometroSin.Stop();` → detiene el cronómetro.

Línea 58: `Console.WriteLine($"Con Tracking: {cronometroCon.ElapsedMilliseconds} ms | Órdenes: {ordenesCon.Count}");` → muestra el tiempo y el número de órdenes.

Línea 59: `Console.WriteLine($"Sin Tracking: {cronometroSin.ElapsedMilliseconds} ms | Órdenes: {ordenesSin.Count}");` → muestra el tiempo y el número de órdenes.

Error común: si se ejecutan las dos consultas en el mismo DbContext, las entidades de la primera consulta permanecen en el Change Tracker cuando se ejecuta la segunda. Se debe usar un DbContext distinto para cada consulta si se quiere medir el tracking de forma aislada.

| Paso 5: Registrar el caso de uso en el contenedor

Modificar `src/AceriaData.Console/Program.cs`:

```
services.AddScoped<TrackingUseCase>();
```

Línea 1: `services.AddScoped();` → registra el caso de uso con ciclo de vida Scoped.

Error común: si se registra con Singleton, se comparte la misma instancia del DbContext entre todos los ámbitos y se producen problemas de concurrencia.

| Paso 6: Llamar al caso de uso desde la consola

Modificar el método Main:

```
using (var scope = _provider.CreateScope())
{
    var useCase = scope.ServiceProvider.GetRequiredService<TrackingUseCase>();
    useCase.Ejecutar();
}
```

Línea 1: `using (var scope = _provider.CreateScope())` → crea un ámbito.

Línea 3: `var useCase = scope.ServiceProvider.GetRequiredService();` → resuelve el caso de uso.

Línea 4: `useCase.Ejecutar();` → ejecuta el caso de uso.

Error común: si se resuelve el caso de uso desde el proveedor raíz en lugar de desde un ámbito, se lanza una excepción `InvalidOperationException`.

| Paso 7: Insertar datos de prueba con varias órdenes

Asegurarse de que hay datos en la base de datos. Modificar el método `Main` para insertar órdenes:

```
using (var scope = _provider.CreateScope())
{
    var context = scope.ServiceProvider.GetRequiredService<AceriaDbContext>();
    context.Database.EnsureDeleted();
    context.Database.Migrate();

    var ordenes = new List<OrdenFabricacion>
    {
        new OrdenFabricacion { NumeroOrden = "OF-2024-0001", Cliente = "Constructora del Norte", Estado = "Pendiente", FechaCreacion = new DateTime(2024, 1, 15) },
        new OrdenFabricacion { NumeroOrden = "OF-2024-0002", Cliente = "Constructora del Sur", Estado = "Pendiente", FechaCreacion = new DateTime(2024, 2, 20) },
        new OrdenFabricacion { NumeroOrden = "OF-2024-0003", Cliente = "Constructora del Este", Estado = "EnProceso", FechaCreacion = new DateTime(2024, 3, 10) },
        new OrdenFabricacion { NumeroOrden = "OF-2024-0004", Cliente = "Constructora del Oeste", Estado = "Pendiente", FechaCreacion = new DateTime(2024, 4, 5) },
        new OrdenFabricacion { NumeroOrden = "OF-2024-0005", Cliente = "Constructora del Norte", Estado = "Pendiente", FechaCreacion = new DateTime(2024, 5, 12) }
    };

    context.OrdenesFabricacion.AddRange(ordenes);
    context.SaveChanges();
}
```

Línea 1: `using (var scope = _provider.CreateScope())` → crea un ámbito.

Línea 3: `var context = scope.ServiceProvider.GetRequiredService();` → resuelve el `DbContext`.

Línea 4: `context.Database.EnsureDeleted();` → elimina la base de datos.

Línea 5: `context.Database.Migrate();` → aplica las migraciones.

Línea 7: `var ordenes = new List` → crea la lista de órdenes.

Línea 9: `new OrdenFabricacion { ... }` → primera orden.

Línea 10: `new OrdenFabricacion { ... }` → segunda orden.

Línea 11: `new OrdenFabricacion { ... }` → tercera orden.

Línea 12: `new OrdenFabricacion { ... }` → cuarta orden.

Línea 13: `new OrdenFabricacion { ... }` → quinta orden.

Línea 16: `context.OrdenesFabricacion.AddRange(ordenes);` → registra las órdenes.

Línea 17: `context.SaveChanges();` → inserta las órdenes.

Error común: si se insertan las órdenes con el mismo número, la clave alternativa rechaza la inserción.

Paso 8: Ejecutar el proyecto

```
dotnet run
```

dotnet run → compila y ejecuta el proyecto.

Resultado esperado: aparecen las secciones con los resultados de cada demostración. Se observa el número de entidades rastreadas con y sin Tracking, y la comparación de tiempos.

Paso 9: Analizar la salida

La salida del programa muestra información como la siguiente:

```
=== TRACKING Y NO TRACKING ===

--- Con Tracking ---
Órdenes cargadas: 5
Entidades rastreadas: 5

--- Sin Tracking ---
Órdenes cargadas: 5
Entidades rastreadas: 5

--- Comparación de rendimiento ---
Con Tracking: 45 ms | Órdenes: 5
Sin Tracking: 38 ms | Órdenes: 5
```

La primera sección muestra que con Tracking se cargan cinco órdenes y se rastrean cinco entidades. La segunda sección muestra que sin Tracking se cargan cinco órdenes pero se rastrean cinco entidades porque el DbContext ya tenía las entidades de la consulta anterior. La tercera sección muestra que sin Tracking es ligeramente más rápido.

Observaciones: el número de entidades rastreadas en la segunda sección es cinco porque el DbContext es el mismo que en la primera sección. Para medir el tracking de forma aislada, se debe usar un DbContext distinto. La diferencia de tiempo es pequeña porque la tabla tiene pocas filas. En tablas con muchas filas, la diferencia es mayor.

Paso 10: Diagnosticar un error común

Modificar el método DemostrarSinTracking para usar AsNoTracking en un DbContext distinto:

```
private void DemostrarSinTrackingAislado()
{
    Console.WriteLine("\n--- Sin Tracking (DbContext aislado) ---");

    using var scope = _provider.CreateScope();
    var repositorio = scope.ServiceProvider.GetRequiredService<IOrdenRepositorio>();

    var ordenes = repositorio.ObtenerSinTracking();
    var rastreadas = repositorio.ContarEntidadesRastreadas();

    Console.WriteLine($"Órdenes cargadas: {ordenes.Count}");
    Console.WriteLine($"Entidades rastreadas: {rastreadas}");
}
```

Línea 1: private void DemostrarSinTrackingAislado() → declara el método.

Línea 3: Console.WriteLine("\n--- Sin Tracking (DbContext aislado) ---"); → muestra la cabecera.

Línea 5: using var scope = _provider.CreateScope(); → crea un ámbito nuevo.

Línea 6: var repositorio = scope.ServiceProvider.GetRequiredService(); → resuelve el repositorio con un DbContext nuevo.

Línea 8: var ordenes = repositorio.ObtenerSinTracking(); → carga las órdenes sin Tracking.

Línea 9: `var rastreadas = repositorio.ContarEntidadesRastreadas();` → cuenta las entidades rastreadas.

Línea 11: `Console.WriteLine($"Órdenes cargadas: {ordenes.Count}");` → muestra el número de órdenes.

Línea 12: `Console.WriteLine($"Entidades rastreadas: {rastreadas}");` → muestra el número de entidades rastreadas.

Resultado esperado con la solución: el número de entidades rastreadas es cero porque el DbContext es nuevo y las entidades no se han registrado.

Errores comunes del ejercicio

Error	Causa	Solución
Cambios no guardados	Se aplicó <code>AsNoTracking</code> y se modificó la entidad	Usar Tracking en consultas de escritura
Número de entidades incorrecto	Se usó el mismo DbContext para varias consultas	Usar un DbContext distinto por consulta
Caché de identidad no usada	Se aplicó <code>AsNoTracking</code>	Usar Tracking si se necesita la caché
Proyección con Tracking	Se aplicó Tracking a una proyección	Las proyecciones no registran entidades
Include con Tracking	Se incluyó una colección con Tracking	El tracking incluye las entidades relacionadas
<code>AsSplitQuery</code> con Tracking	Se dividió la consulta con Tracking	El tracking se aplica a todas las consultas

Reto resuelto: Comparar tracking en consultas con Include

Reto: Crear dos métodos en el repositorio que carguen las órdenes con sus planchas, uno con Tracking y otro sin Tracking. Comparar el número de entidades rastreadas en cada caso.

Solución paso a paso:

Paso 1: Añadir los métodos a la interfaz `IOrdenRepositorio`:

```
List<OrdenFabricacion> ObtenerConPlanchasConTracking();
List<OrdenFabricacion> ObtenerConPlanchasSinTracking();
```

Línea 1: `List ObtenerConPlanchasConTracking();` → declara el método con Tracking.

Línea 2: `List ObtenerConPlanchasSinTracking();` → declara el método sin Tracking.

Paso 2: Implementar los métodos en OrdenRepositorio:

```
public List<OrdenFabricacion> ObtenerConPlanchasConTracking()
{
    return _context.OrdenesFabricacion
        .Include(o => o.Planchas)
        .OrderBy(o => o.NumeroOrden)
        .ToList();
}

public List<OrdenFabricacion> ObtenerConPlanchasSinTracking()
{
    return _context.OrdenesFabricacion
        .AsNoTracking()
        .Include(o => o.Planchas)
        .OrderBy(o => o.NumeroOrden)
        .ToList();
}
```

Línea 1: public List ObtenerConPlanchasConTracking() → declara el método con Tracking.

Línea 3: return _context.OrdenesFabricacion → inicia la consulta.

Línea 4: .Include(o => o.Planchas) → incluye la colección de planchas.

Línea 5: .OrderBy(o => o.NumeroOrden) → ordena por número de orden.

Línea 6: .ToList(); → materializa la consulta con Tracking.

Línea 9: public List ObtenerConPlanchasSinTracking() → declara el método sin Tracking.

Línea 11: return _context.OrdenesFabricacion → inicia la consulta.

Línea 12: .AsNoTracking() → aplica AsNoTracking.

Línea 13: .Include(o => o.Planchas) → incluye la colección de planchas.

Línea 14: .OrderBy(o => o.NumeroOrden) → ordena por número de orden.

Línea 15: .ToList(); → materializa la consulta sin Tracking.

Paso 3: Añadir la demostración en el caso de uso:

```
private void DemostrarTrackingConInclude()
{
    Console.WriteLine("\n--- Tracking con Include ---");

    using var scopeCon = _provider.CreateScope();
    var repoCon = scopeCon.ServiceProvider.GetRequiredService<IOrdenRepositorio>();
    var ordenesCon = repoCon.ObtenerConPlanchasConTracking();
    var rastreadasCon = repoCon.ContarEntidadesRastreadas();
    Console.WriteLine($"Con Tracking: órdenes: {ordenesCon.Count} | entidades rastreadas: {rastreadasCon}");

    using var scopeSin = _provider.CreateScope();
    var repoSin = scopeSin.ServiceProvider.GetRequiredService<IOrdenRepositorio>();
    var ordenesSin = repoSin.ObtenerConPlanchasSinTracking();
    var rastreadasSin = repoSin.ContarEntidadesRastreadas();
    Console.WriteLine($"Sin Tracking: órdenes: {ordenesSin.Count} | entidades rastreadas: {rastreadasSin}");
}
```

Línea 1: private void DemostrarTrackingConInclude() → declara el método.

Línea 3: Console.WriteLine("\n--- Tracking con Include ---"); → muestra la cabecera.

Línea 5: using var scopeCon = _provider.CreateScope(); → crea un ámbito para la consulta con Tracking.

Línea 6: var repoCon = scopeCon.ServiceProvider.GetRequiredService(); → resuelve el repositorio.

Línea 7: `var ordenesCon = repoCon.ObtenerConPlanchasConTracking();` → carga las órdenes con Tracking.

Línea 8: `var rastreadasCon = repoCon.ContarEntidadesRastreadas();` → cuenta las entidades rastreadas.

Línea 9: `Console.WriteLine($"Con Tracking: órdenes: {ordenesCon.Count} | entidades rastreadas: {rastreadasCon}");` → muestra los resultados.

Línea 11: `using var scopeSin = _provider.CreateScope();` → crea un ámbito para la consulta sin Tracking.

Línea 12: `var repoSin = scopeSin.ServiceProvider.GetRequiredService();` → resuelve el repositorio.

Línea 13: `var ordenesSin = repoSin.ObtenerConPlanchasSinTracking();` → carga las órdenes sin Tracking.

Línea 14: `var rastreadasSin = repoSin.ContarEntidadesRastreadas();` → cuenta las entidades rastreadas.

Línea 15: `Console.WriteLine($"Sin Tracking: órdenes: {ordenesSin.Count} | entidades rastreadas: {rastreadasSin}");` → muestra los resultados.

Paso 4: Llamar al método desde Ejecutar:

```
DemostrarTrackingConInclude();
```

Paso 5: Ejecutar dotnet run y comparar los resultados.

Resultado esperado: con Tracking, el número de entidades rastreadas incluye las órdenes y las planchas. Sin Tracking, el número de entidades rastreadas es cero.

Analogía final

El tracking en una acería es como el libro de producción que el supervisor mantiene durante el turno. Cuando una plancha entra en la línea, el supervisor la anota en el libro con todos sus datos. Si la plancha se modifica, el supervisor lo anota. Al final del turno, el supervisor revisa el libro y genera las órdenes de actualización. El tracking consume tiempo y espacio: cada plancha anotada ocupa una línea en el libro. El no tracking es como no anotar las planchas en el libro: si solo se van a consultar, no hace falta anotarlas. El libro se mantiene limpio y el supervisor se ahorra trabajo. Pero si se quiere modificar una plancha, hay que anotarla primero. `AsNoTracking` es como decirle al supervisor que no anote las planchas: se cargan más rápido, pero no se pueden modificar. `AsTracking` es como decirle al supervisor que anote las planchas aunque el turno sea de solo consulta. La caché de identidad es como el fichero de planchas ya anotadas: si se pide la misma plancha dos veces, el supervisor devuelve la misma ficha. Sin tracking, el supervisor crea una ficha nueva cada vez. Así funciona el tracking en EF Core: se anota lo que se va a modificar, se deja de anotar lo que solo se va a consultar.

Resultado esperado

Al final del ejercicio, deberías haber:

Añadido los métodos de tracking a la interfaz `IOrdenRepositorio`.

Implementado los métodos en `OrdenRepositorio`.

Creado el caso de uso `TrackingUseCase`.

Registrado el caso de uso en el contenedor.

Insertado datos de prueba con varias órdenes.

Ejecutado las demostraciones de Tracking, No Tracking y comparación de rendimiento.

Analizado la salida y comprobado el número de entidades rastreadas.

Diagnosticado el error de usar el mismo `DbContext` para varias consultas.

Creado los métodos `ObtenerConPlanchasConTracking` y `ObtenerConPlanchasSinTracking`.